

Sorabel MCP gateway - Guide d'accès

Page **générée** le 2026-09-13 par `make doc-tools`, depuis ce que `tools/list` répond réellement, profil par profil. Elle n'est pas rédigée : un argument renommé ou un tool retiré à un profil s'y voit à la régénération suivante.

Cette page se suffit à elle-même : se brancher, lire une réponse, et les huit fiches. Ce qu'elle ne porte pas — les garanties détaillées, les limites chiffrées et les écarts assumés — vit dans le guide d'intégration du dépôt.

1 · Se brancher

Le serveur parle **stdio** : ni port, ni URL, ni jeton. Un processus par profil, et le profil est posé par **qui lance le processus** — jamais par le client, jamais par le modèle.

Prérequis

Python `>=3.11, <3.12`, [uv](#), et Docker — ce dernier pour la seule base vectorielle.

```
uv sync --extra vector          # l'extra est nécessaire : sans lui l'embedder local cass
cp .env.example .env           # aucune valeur du modèle n'est un secret
make up                         # Chroma, via docker compose, port 8002
make seed                       # génère data/sorabel.db
make ingest                     # indexe le corpus (~400 éditions)
```

`.env` n'est pas obligatoire : les **26 réglages** ont **tous** un défaut. Il le devient pour les appels de modèle — `AZURE_AI_ENDPOINT`, `AZURE_AI_API_KEY`, `LLM_CHAT_MODEL` — dont dépendent `answer_question` (la rédaction) et `ask_database` (la génération SQL). **Les six autres tools fonctionnent sans** : la recherche et le reclassement tournent en local par défaut.

Le bloc de configuration

À coller dans la configuration de votre client MCP :

```
{
  "mcpServers": {
    "sorabel": {
      "command": "uv",
      "args": ["run", "python", "-m", "mcp_server.server"],
      "cwd": "/chemin/absolu/vers/brief22-dev-phase_sorabel-gateway",
      "env": {
        "SORABEL_PROFILE": "commercial",
```

```

    "TOKENIZERS_PARALLELISM": "false"
  }
}
}
}

```

`cwd` est **obligatoire** : le corpus, la base et la matrice sont résolus relativement à la racine du dépôt. `TOKENIZERS_PARALLELISM` n'est pas fonctionnel — il fait taire un avertissement de la bibliothèque d'embeddings qui, sinon, encombre la console du client. `SORABEL_PROFILE` vaut `support` par défaut ; un nom inconnu retombe sur `default`, qui n'a aucun tool — un catalogue vide est donc le symptôme d'un profil mal orthographié, pas d'une panne.

Préférez ce bloc à une commande en ligne de commande. Certains clients — l'Inspector officiel en fait partie — analysent les arguments et **avalent le `-m`**, ce qui lance `python` sans module : le processus lit alors le JSON-RPC sur son entrée standard et le prend pour du code. Le symptôme est un `NameError: name 'true' is not defined` suivi d'un délai d'attente dépassé.

Vérifier que ça répond

```

make serve                # le serveur, en stdio, profil dans SORABEL_PROFILE
make client               # un client de test : le catalogue du profil
PROFILE=commercial make client

# avec l'Inspector officiel, le bloc ci-dessus enregistré dans mon-serveur.json :
npx @modelcontextprotocol/inspector --cli \
  --config mon-serveur.json --server sorabel --method tools/list --format json

```

2 · Profil × tools — l'étage 1, tel que le serveur l'applique

Chaque colonne est un catalogue réellement obtenu, en lançant le serveur avec `SORABEL_PROFILE` à cette valeur. Un profil ne voit pas seulement moins de tools : les **descriptions** qu'il reçoit diffèrent aussi — voir les fiches.

Tool	default	dev	support	commercial	admin
<code>answer_question</code>	.	✓	✓	✓	✓
<code>search_docs</code>	.	✓	✓	✓	✓
<code>get_document</code>	.	✓	✓	✓	✓
<code>list_sources</code>	.	✓	✓	✓	✓
<code>ask_database</code>	.	.	✓	✓	✓
<code>get_schema</code>	.	✓	.	✓	✓

check_stock	.	.	✓	✓	✓
order_status	.	.	✓	✓	✓
total du catalogue	0	5	7	8	8
read_journal hors protocole	.	.	.	.	✓

Deux colonnes identiques ne veulent pas dire deux profils identiques. La matrice accorde `read_journal` (admin) — un droit qui n'est **pas** un tool MCP : il n'est publié par aucun `tools/list`, et s'exerce par l'API du banc d'essai. Deux profils au même catalogue peuvent donc différer par là.

3 • Lire une réponse

Les huit tools rendent la **même** enveloppe, servie comme refusée. Ce qui suit est lu dans l' `outputSchema` que le serveur publie — c'est le texte que reçoit déjà votre client, pas une paraphrase.

Enveloppe de réponse de la gateway Sorabel. Le discriminant est « status », puis « payload.code » qui distingue les cas d'un même statut. Ne jamais rendre à l'utilisateur un payload dont la clé de charge utile est absente : c'est un refus ou une non-réponse, et il n'y a rien à afficher.

Les trois clés sont **toujours** présentes : `status`, `payload`, `message`.

Clé	Ce que le serveur en dit
status	L'issue de l'appel. « error » : l'exécution a échoué. « hors_corpus » : le corpus ne porte pas la réponse — ce n'est pas un refus. « ok » : servi. « refused » : la gouvernance a tranché.
payload	Objet. Toujours présent : <code>code</code> . Les autres clés dépendent du tool — voir sa fiche — et sont absentes , jamais vides, quand il n'y a rien à mettre dedans.
message	Phrase figée choisie sur le code, destinée à l'utilisateur. Vide sur « ok », non vide sur tout autre statut. Elle ne contient jamais de texte de modèle, de message de moteur ni de nom d'interne.

Les 12 codes

Le `status` dit l'issue, le `code` dit le cas précis. **Branchez sur `code`** : deux codes très différents partagent parfois un statut. Quatre codes seulement sont des refus — les autres sont des constats, et les traiter comme des erreurs surestimerait la sévérité du système.

Code	Statut	Domaine	Refus ?
ambiguite_donnees	ok	base	non

aucune_ligne	ok	base et documentaire	non
clarification	clarification	base	non
contexte_insuffisant	hors_corpus	documentaire	non
ecriture_refusee	refused	base	oui
erreur_execution	error	base et documentaire	non
hors_corpus	hors_corpus	documentaire	non
hors_schema	refused	base	oui
introuvable	ok	documentaire	non
ok	ok	base et documentaire	non
perimetre_interdit	refused	base et documentaire	oui
tool_interdit	refused	base et documentaire	oui

Deux réponses réelles

Obtenues en appelant le serveur pendant la génération de cette page, pas recopiées.

CHECK_STOCK {"REFERENCE": "REF-8842"} — PROFIL SUPPORT

Un résultat — la charge utile est là, message est vide.

```
{
  "status": "ok",
  "payload": {
    "code": "ok",
    "sql": "SELECT s.entrepot, s.quantite, s.seuil_reappro, CASE WHEN s.quantite < s.se",
    "columns": [
      "entrepot",
      "quantite",
      "seuil_reappro",
      "sous_seuil"
    ],
    "rows": [
      [
        "LILLE",
        247,
        10,
        0
      ],
      [
        "LYON",
        100,
        50,
        0
      ]
    ]
  }
}
```

```
[
  "NANTES",
  427,
  20,
  0
],
"total": 774,
"reference": "REF-8842"
},
"message": ""
}
```

GET_SCHEMA {} — PROFIL SUPPORT

Un refus — `support` n'a pas ce tool. **La clé de charge utile n'est pas vide : elle est absente.** Il n'y a rien à afficher, et c'est ce qui empêche un client de rendre un refus comme une réponse.

```
{
  "status": "refused",
  "payload": {
    "code": "tool_interdit"
  },
  "message": "Cette information n'est pas accessible avec votre profil."
}
```

4 • Les 8 tools

answer_question

Réponse documentaire sourcée

Visible par : `dev` `support` `commercial` `admin`

ARGUMENTS

Argument	Type	Présence	Contrainte
<code>question</code>	string	requis	—
<code>collections</code>	array	optionnel	tableau de string

RÉPONSE

STATUTS POSSIBLES `error` · `hors_corpus` · `ok` · `refused`

CLÉS DE CHARGE UTILE `answer` · `sources` absentes sur un refus ou une non-réponse

DESCRIPTION SERVICE — ELLE DIFFÈRE SELON LE PROFIL

Servie à : dev

OBJET	répondre à une question sur la documentation interne Sorabel, en citant les documents utilisés.
ENTRÉE	`question`, en langage naturel. `collections` restreint aux types de documents voulus, et peut être omis.
SORTIE	une réponse rédigée et la liste de ses sources (référence, titre) ; ou une non-réponse quand le corpus ne porte pas de quoi répondre.
UTILISER QUAND	la question porte sur une procédure, une consigne interne, une notice, ou une caractéristique de produit.
NE PAS UTILISER QUAND	la question attend un chiffre ou un état tenu par la base — la documentation décrit, elle ne compte pas. Pour les extraits bruts sans rédaction, utiliser <code>search_docs</code> .

Servie à : support, commercial, admin — seules les rubriques ci-dessous changent

NE PAS UTILISER QUAND	la question attend un chiffre ou un état tenu par la base — la documentation décrit, elle ne compte pas. Pour les extraits bruts sans rédaction, utiliser <code>search_docs</code> . Pour l'état d'un produit et non sa description — stock, chiffres, commandes —, utiliser <code>ask_database</code> . Un seul cas demande DEUX appels, et c'est le seul : une référence donnée SEULE, sans question — « REF-5313 ». Appeler alors AUSSI <code>check_stock</code> , et rendre la fiche ET le stock. Hors de ce cas, un seul tool suffit.
-----------------------	--

Annotations : `readOnlyHint` · `idempotentHint`

`search_docs`

Recherche d'extraits documentaires

Visible par : `dev` `support` `commercial` `admin`

ARGUMENTS

Argument	Type	Présence	Contrainte
<code>query</code>	string	requis	—
<code>collections</code>	array	optionnel	tableau de string

RÉPONSE

STATUTS POSSIBLES `error` · `hors_corpus` · `ok` · `refused`

CLÉS DE CHARGE UTILE

`hits` absentes sur un refus ou une non-réponse

DESCRIPTION SERVICE

OBJET	rendre les extraits documentaires les plus pertinents pour une recherche, sans rédiger de réponse.
ENTRÉE	`query`, mots-clés ou question. `collections` restreint aux types de documents voulus, et peut être omis.
SORTIE	les extraits classés par pertinence, avec leurs métadonnées — identifiant d'édition, titre, type, thème.
UTILISER QUAND	il faut lire le texte source lui-même, comparer plusieurs passages, ou obtenir l'identifiant d'une édition.
NE PAS UTILISER QUAND	une réponse formulée est attendue, plutôt que des extraits. Pour une réponse rédigée et sourcée, utiliser <code>answer_question</code> . Pour le texte intégral d'une édition identifiée, utiliser <code>get_document</code> .

Annotations : `readOnlyHint` · `idempotentHint`

`get_document`

Texte intégral d'un document

Visible par : `dev` `support` `commercial` `admin`

ARGUMENTS

Argument	Type	Présence	Contrainte
<code>doc_id</code>	string	requis	—
<code>version</code>	string	optionnel	—

RÉPONSE

STATUTS POSSIBLES `error` · `hors_corpus` · `ok` · `refused`

CLÉS DE CHARGE UTILE `text` · `metadata` absentes sur un refus ou une non-réponse

DESCRIPTION SERVICE

OBJET	rendre le texte intégral d'une édition documentaire déjà identifiée.
ENTRÉE	`doc_id`, un identifiant d'édition de la forme `notices/notice-REF-1589-v1.0`. `version` peut être omis.
SORTIE	le texte complet et les métadonnées de l'édition ; ou une non-réponse quand aucune édition ne porte cet identifiant.

UTILISER QUAND

l'identifiant d'édition est déjà connu, parce qu'un autre tool

documentaire vient de le rendre.

NE PAS UTILISER QUAND

l'entrée est une question en langage naturel, ou une référence produit de la forme REF-NNNN — une référence produit n'est pas un identifiant d'édition, et ce tool ne la trouvera pas.

L'identifiant attendu est celui que rend `search_docs`.

`list_sources` en donne l'inventaire. Pour une question, ou pour une référence produit REF-NNNN, utiliser `answer_question`.

Annotations : `readOnlyHint` · `idempotentHint`

`list_sources`

Inventaire du corpus documentaire

Visible par : `dev` `support` `commercial` `admin`

ARGUMENTS

Argument	Type	Présence	Contrainte
<code>collections</code>	array	optionnel	tableau de string

RÉPONSE

STATUTS POSSIBLES `error` · `hors_corpus` · `ok` · `refused`

CLÉS DE CHARGE UTILE `sources` absentes sur un refus ou une non-réponse

DESCRIPTION SERVIE

OBJET inventorer les documents accessibles, sans déclencher de recherche.

ENTRÉE ``collections`` restreint aux types de documents voulus, et peut être omis.

SORTIE la liste des documents — identifiant, titre, type — et leur nombre total.

UTILISER QUAND il faut savoir ce que le corpus couvre avant de poser une question, ou récupérer des identifiants d'éditions.

NE PAS UTILISER QUAND la question porte sur le contenu des documents et non sur leur existence. Pour chercher, utiliser `search_docs`. Pour une réponse rédigée, utiliser `answer_question`.

Annotations : `readOnlyHint` · `idempotentHint`

ask_database

Interrogation de la base métier

Visible par : support commercial admin

ARGUMENTS

Argument	Type	Présence	Contrainte
question	string	requis	—

RÉPONSE

STATUTS POSSIBLES clarification · error · ok · refused

CLÉS DE CHARGE UTILE sql · columns · rows · conventions · truncated · axes absentes sur un refus ou une non-réponse

DESCRIPTION SERVIE

OBJET	traduire une question chiffrée en une lecture SQL de la base commerciale, et l'exécuter.
ENTRÉE	`question`, en langage naturel.
SORTIE	les colonnes et les lignes du résultat, la requête SQL exécutée et les conventions métier appliquées ; ou un refus quand la question sort du périmètre du profil.
UTILISER QUAND	la question demande un décompte, une somme, une moyenne, un classement, un filtre ou une période sur les produits, stocks, commandes, clients ou ventes.
NE PAS UTILISER QUAND	la réponse attendue est une procédure ou une caractéristique décrite dans un document. Pour le stock d'une référence, utiliser <code>check_stock</code> . Pour une commande précise, utiliser <code>order_status</code> .

Annotations : readOnlyHint · idempotentHint

get_schema

Schéma SQL autorisé

Visible par : dev commercial admin

ARGUMENTS

Aucun argument.

RÉPONSE

STATUTS POSSIBLES `clarification` · `error` · `ok` · `refused`

CLÉS DE CHARGE UTILE `schema` absentes sur un refus ou une non-réponse

DESCRIPTION SERVIE — ELLE DIFFÈRE SELON LE PROFIL

Servie à : dev

OBJET	rendre lisible la forme de la base — tables, colonnes, énumérations, conventions métier.
ENTRÉE	aucun argument.
SORTIE	le schéma des seules tables et colonnes du périmètre, et la plage de dates couverte.
UTILISER QUAND	il faut savoir ce que la base contient avant de formuler une question, ou écrire du code contre elle.
NE PAS UTILISER QUAND	la question attend une valeur, un décompte ou des lignes — ce tool ne lit aucune donnée, et le schéma seul ne répond à aucune question chiffrée.

Servie à : commercial, admin — seules les rubriques ci-dessous changent

NE PAS UTILISER QUAND	la question attend une valeur, un décompte ou des lignes — ce tool ne lit aucune donnée, et le schéma seul ne répond à aucune question chiffrée. Pour obtenir un résultat, utiliser <code>ask_database</code> .
-----------------------	---

Annotations : `readOnlyHint` · `idempotentHint`

check_stock

Stock d'une référence

Visible par : `support` `commercial` `admin`

ARGUMENTS

Argument	Type	Présence	Contrainte
<code>reference</code>	string	requis	<code>^REF-\d{4}\$</code>

RÉPONSE

STATUTS POSSIBLES `clarification` · `error` · `ok` · `refused`

CLÉS DE CHARGE UTILE `sql` · `columns` · `rows` · `total` · `reference` · `truncated` absentes sur un refus ou une non-réponse

DESCRIPTION SERVICE

OBJET	rendre le stock d'une référence produit, entrepôt par entrepôt.
ENTRÉE	`reference`, de la forme REF-NNNN exactement.
SORTIE	une ligne par entrepôt — quantité et seuil de réapprovisionnement ; ou une non-réponse quand la référence n'existe pas.
UTILISER QUAND	la question porte sur la disponibilité ou la quantité d'une référence connue.
NE PAS UTILISER QUAND	l'entrée est un nom de produit et non une référence, ou la question porte sur ce qu'est le produit — ce tool ne rend que des quantités, jamais une caractéristique. Pour les caractéristiques d'une référence — fiche technique, notice, procédure —, utiliser <code>answer_question</code> . Un seul cas demande DEUX appels, et c'est le seul : une référence donnée SEULE, sans question — « REF-5313 ». Appeler alors AUSSI <code>answer_question</code> , et rendre la fiche ET le stock. Hors de ce cas, un seul tool suffit. Pour une autre question chiffrée, utiliser <code>ask_database</code> .

Annotations : `readOnlyHint` · `idempotentHint`

order_status

Statut d'une commande

Visible par : `support` `commercial` `admin`

ARGUMENTS

Argument	Type	Présence	Contrainte
<code>order_id</code>	string	requis	<code>^CMD-\d{4}-\d{4}\$</code>

RÉPONSE

STATUTS POSSIBLES `clarification` · `error` · `ok` · `refused`

CLÉS DE CHARGE UTILE `sql` · `columns` · `rows` · `order_id` · `truncated` absentes sur un refus ou une non-réponse

DESCRIPTION SERVICE

OBJET	rendre l'en-tête d'une commande.
ENTRÉE	`order_id`, de la forme CMD-AAAA-NNNN exactement.
SORTIE	le client, la date, le statut et le montant HT de la commande ;

UTILISER QUAND

NE PAS UTILISER QUAND

ou une non-réponse quand l'identifiant n'existe pas.

la question porte sur une commande unique et identifiée.

plusieurs commandes sont visées, ou un agrégat est demandé. Pour plusieurs commandes ou un agrégat, utiliser `ask_database`.

Annotations : `readOnlyHint` · `idempotentHint`

Généré par `scripts/build_tool_docs.py` — `initialize` , `tools/list` , et les deux appels dont les réponses sont montrées plus haut. Aucun appel de modèle. Le contrat de réponse est déclaré dans `mcp_server/output_schemas.py` , la matrice dans `mcp_server/matrice.yaml` .